


# Team Project 1 - CAP

## CSS Fundamental

Amjad



# Introduction to CSS

CSS (Cascading Style Sheets) is the language used to describe the presentation of a web page and web elements . It controls the **appearance and layout** of HTML web pages.

It defines how elements should look, including colors, fonts, spacing, and layouts. CSS separates **content (HTML)** from **design (CSS)**, which makes websites easier to manage and more consistent.

**Why important:**

- 🎨 Change **colors** and backgrounds
- 🔤 Control **fonts**, text sizes, and styles
- 📐 Add **spacing, borders, and sizing**
- 🧱 Control the **layout and positioning** of elements
- 📱 Create **responsive designs** for different screen sizes
- ✨ Add **animations and visual effects**

Anjad

# CSS syntax

CSS uses **rules** to define how HTML elements should look. Every CSS rule follows a structure:

- **Selector:** Targets the HTML element(s).
- **Property:** The aspect to style (e.g., color, font-size).
- **Value:** The setting (e.g., red, 20px).

**Why important:**

Understanding syntax is the foundation of writing CSS correctly.

```
selector {
    property: value;
    property: value;
}
```

**Important:**

- A property and its value are separated by a **colon :**
- Each declaration normally ends with a **semicolon ;**
- Declarations are placed inside **curly braces { }**

Amjad

# CSS selectors

A **CSS selector** is used to **select the HTML element(s)** that you want to style. Selectors define which HTML elements the style applies to.

## Common CSS Selectors

| Selector | Example | Selects |
|---|---|---|
| **Element** | p | All <p> elements |
| **Class** | .note | Elements with class="note" |
| **ID** | #header | The element with id="header" |
| **Universal** | * | All elements |
| **Grouping** | h1, h2, p | All <h1>, <h2>, and <p> elements |

```css
/* Element selector */
p {
    color: blue;
}

/* Class selector */
.note {
    color: green;
}

/* ID selector */
#header {
    background-color: gray;
}

/* Universal selector */
* {
    font-family: Arial;
}

/* Grouping selector */
h1, h2, p {
    text-align: center;
}
```

**Why important:**

Allows precise control over styling without repeating code.

# How to use/apply CSS

There are **three main ways** to apply CSS to an HTML page:

1. **Inline CSS:** Inside an element (style="").
2. **Internal CSS:** Inside <style> in the <head>.
3. **External CSS:** Linked CSS file (.css).

**Which Should You Use?**
**External CSS is generally preferred** because it keeps HTML and styling separate, makes CSS reusable across multiple pages, and is easier to maintain.

Amjad

# How to use/apply CSS

**1. Inline CSS:** CSS is written directly inside an HTML element using the style attribute.

```
<p style="color: blue;">Hello World</p>
```

**Best for:** Quick or one-time styling.

Amjad

# How to use/apply CSS

**2. Internal CSS:** CSS is written inside a <style> element in the HTML <head>.

```
<head>
    <style>
        p {
            color: blue;
        }
    </style>
</head>
```

**Best for:** Small or single-page websites.

Anjad

# How to use/apply CSS

**3. External CSS:** CSS is written in a separate .css file and linked to the HTML document.

HTML

```html
<link rel="stylesheet" href="style.css">
```

**style.css:**

CSS

```css
p {
    color: blue;
}
```

**Best for:** Larger websites with multiple pages.

# CSS Comments

- CSS comments are used to add **notes or explanations** inside CSS code.
- Comments are **ignored by the browser** and do not affect the page.

**Why use comments?**

- Explain CSS code
- Organize styles into sections
- Temporarily disable CSS declarations
- Make code easier to understand and maintain

```css
/* This is a CSS comment */

/* Style the main heading */
h1 {
    color: blue;
    font-size: 32px;
}

/* Set the page background */
body {
    background-color: lightgray;
}

/*
   This is a
   multi-line comment
*/
```

Amir

# CSS Fonts

Fonts control how text appears on a web page and affect **readability, appearance, and brand identity**.
CSS can use:

- **System fonts**: fonts already installed on the user's device.
- **Web fonts**: custom fonts loaded from services such as Google Fonts or Adobe Fonts.

**Main Font Properties**

| Property | Purpose | Example |
|---|---|---|
| font-family | Specifies the font | Arial, sans-serif |
| font-size | Controls text size | 18px |
| font-weight | Controls text thickness | bold or 700 |
| line-height | Controls spacing between lines | 1.5 |

```css
p {
    font-family: Arial, sans-serif;
    font-size: 18px;
    font-weight: 400;
    line-height: 1.5;
}
```

It is good practice to provide alternative fonts:

```css
font-family: Arial, Helvetica, sans-serif;
```

The browser tries **Arial** first, then **Helvetica**, and finally uses the device's default **sans-serif** font.

# CSS Fonts

**Web fonts must be loaded/imported before you can use them in CSS.** The exact method depends on the provider.

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap"
      rel="stylesheet">
```

Then use it in CSS:

```
body {
    font-family: "Roboto", sans-serif;
}
```

Google Fonts also provides a CSS @import option:

```
@import url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap');

body {
    font-family: "Roboto", sans-serif;
}
```

# CSS Pseudo-classes

- A **CSS pseudo-class** is a keyword added to a selector to style an element based on a **special state or condition**, without adding or changing HTML classes.
- Pseudo-classes start with a **colon :**.

**Common Pseudo-classes**

| Pseudo-class | Description |
|---|---|
| :hover | When the mouse pointer is over an element |
| :focus | When an element receives focus |
| :nth-child(n) | Selects elements based on their position among siblings |

**Syntax**

```
selector:pseudo-class {
    property: value;
}
```

**Pseudo-classes allow CSS to react to element states and conditions without JavaScript or additional HTML classes.**

# CSS Pseudo-classes

CSS

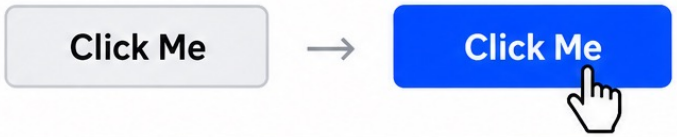
```css
/* Mouse over a button */
button:hover {
    background-color: blue;
    color: white;
}

/* Input receives focus */
input:focus {
    border: 2px solid blue;
}

/* Select the second list item */
li:nth-child(2) {
    color: red;
}
```

Result

1. :hover (mouse over the button)




2. :focus (input receives focus)




3. :nth-child(2) (second list item)

- First item
- Second item
- Third item
- Fourth item



# CSS Pseudo-classes

## Common Examples of Pseudo-classes:

### User Action Pseudo-classes:

- `:hover`: Styles an element when the user's mouse pointer is over it.
- `:active`: Styles an element while it is being activated (e.g., clicked).
- `:focus`: Styles an element when it has received focus (e.g., from tabbing or clicking on an input field).

### Link Pseudo-classes:

- `:link`: Styles unvisited links.
- `:visited`: Styles visited links.

### Structural Pseudo-classes:

- `:first-child`: Styles the first child element of its parent.
- `:last-child`: Styles the last child element of its parent.
- `:nth-child(n)`: Styles elements based on their position among a group of siblings
- `:only-child`: Styles an element if it is the only child of its parent.

### Form Element Pseudo-classes:

- `:checked`: Styles a checked radio button or checkbox.
- `:disabled`: Styles a disabled form element.
- `:enabled`: Styles an enabled form element.
- `:valid`: Styles a form element whose content is valid according to its validation rules.
- `:invalid`: Styles a form element whose content is invalid.

Najah



# CSS Pseudo-elements

A **CSS pseudo-element** is used to style a **specific part of an element** or to insert decorative content **without adding new HTML elements**.
Pseudo-elements are identified by a **double colon ::**.

**Common Pseudo-elements**

| Pseudo-element | Description |
|---|---|
| ::before | Creates content before an element's existing content |
| ::after | Creates content after an element's existing content |
| ::first-letter | Styles the first letter of text |
| ::first-line | Styles the first line of text |

**Syntax**

```
selector::pseudo-element {
    property: value;
}
```

`:pseudo-class` → styles an element based on its **state or condition**
`::pseudo-element` → styles a **part of an element** or creates a generated pseudo-element

# CSS Pseudo-elements

**HTML**

```html
<h2>Important</h2>
```

**CSS**

```css
h2::before {
    content: "★ ";
    color: gold;
}

h2::after {
    content: "★";
    color: gold;
}
```

**Result**

★ Important ★

# CSS Math Functions

- **CSS Math Functions** allow mathematical expressions and comparisons to be used directly inside CSS property values.
- They are useful for creating **dynamic, flexible, and responsive layouts**.

**Common Math Functions**:

| Function | Purpose | Example |
|---|---|---|
| calc() | Performs mathematical calculations | width: calc(100% - 40px); |
| min() | Uses the smallest value | width: min(90%, 800px); |
| max() | Uses the largest value | width: max(50%, 300px); |

**Why Use Math Functions?**

- 📱 Create responsive layouts
- 📏 Mix different units such as %, px, rem, and pt
- 🔄 Adapt sizes automatically to available space

Aniaq

# CSS Math Functions

## calc() Example

```
.container {
    width: calc(100% - 40px);
}
```

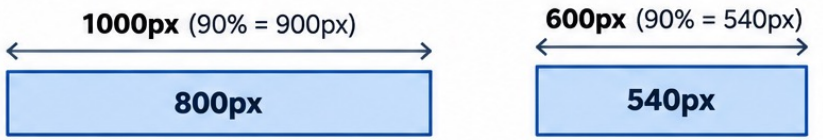



If the available width is **1000px**:

**1000px – 40px = 960px**

## min() Example

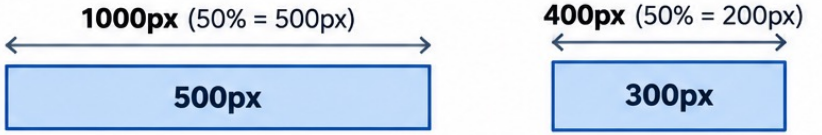
```
.container {
    width: min(90%, 800px);
}
```




The width will be **90%** or **800px**, whichever is **smaller**.

## max() Example

```
.container {
    width: max(50%, 300px);
}
```




The width will be **50%** or **300px**, whichever is **larger**.

# CSS Box Model

Every HTML element is treated as a **box**. The CSS Box Model describes the different areas that surround an element.

**The Four Parts:**

1. **Content** → The actual content of the element, such as text or an image.
2. **Padding** → Space **between the content and the border**.
3. **Border** → The edge that surrounds the content and padding.
4. **Margin** → Space **outside the border**, separating the element from other elements.

```css
.box {
    width: 300px;
    padding: 20px;
    border: 5px solid black;
    margin: 30px;
}
```

**Why important:** Understanding the Box Model is essential for controlling **element size, spacing, and page layout**.

**From Inside → Outside**
**Content → Padding → Border → Margin**

Anjar

# CSS Box Model - Content

## Content

- The **content** is the **innermost area** of the CSS Box Model.
- It contains the actual content of the HTML element, such as:
  - Text
  - Images
  - Videos
  - Other HTML elements

```
.box {
    width: 300px;
    height: 150px;
}
```

**Width and Height**

By default, the CSS width and height properties define the dimensions of the **content area**.

Anjal

# CSS Box Model - Padding

## Padding

**Padding** is the space between an element's **content** and its **border**.
It creates **internal spacing**, preventing the content from sitting directly against the border.

### Padding Property

```css
.box {
    padding: 20px;
}
```

This adds 20px of padding on **all four sides**.

### Individual Sides

```css
.box {
    padding-top: 10px;
    padding-right: 20px;
    padding-bottom: 10px;
    padding-left: 20px;
}
```

### Shorthand

```css
.box {
    padding: 10px 20px;
}
```

**10px** → Top & Bottom
**20px** → Left & Right

**Remember:** Padding creates space **inside the border**.

Amjad

# CSS Box Model - Border

## Border

- The **border** is the line that surrounds an element's **content and padding**.
- It creates a visible boundary around the element.

### Border Property

A border commonly defines:

- **Width** → thickness of the border
- **Style** → solid, dashed, dotted, etc.
- **Color** → color of the border

### Border Property

```
.box {
    border: 3px solid black;
}
```

### Individual Properties

```
.box {
    border-width: 3px;
    border-style: solid;
    border-color: black;
}
```

### Individual Sides

```
.box {
    border-top: 2px solid red;
    border-right: 3px dashed blue;
    border-bottom: 2px solid green;
    border-left: 3px dotted black;
}
```

Amjad

# CSS Box Model - Margin

## Margin

- The **margin** is the space **outside an element's border**.
- It creates separation between the element and **other surrounding elements**.

### Margin Property

```
.box {
    margin: 20px;
}
```

### Individual Sides

```
.box {
    margin-top: 10px;
    margin-right: 20px;
    margin-bottom: 10px;
    margin-left: 20px;
}
```

### Shorthand

```
.box {
    margin: 10px 20px;
}
```

**10px** → Top & Bottom
**20px** → Left & Right

### Centering an Element

```
.box {
    width: 500px;
    margin: 0 auto;
}
```

`auto` left and right margins can be used to **horizontally center a block element with a defined width**.

**Remember:** Margin creates space **outside the border**.

Amjad

# CSS Position

The CSS position property controls **how an element is positioned on the page**.

There are **five main position values**:

| Value | Behavior |
|---|---|
| static | Default position; follows the normal document flow |
| relative | Positioned relative to its **original position** |
| absolute | Positioned relative to its **nearest positioned ancestor** |
| fixed | Positioned relative to the **viewport** and stays in place while scrolling |
| sticky | Acts like relative, then sticks at a specified position while scrolling |

```
.box {
    position: absolute;
    top: 20px;
    right: 30px;
}
```

# CSS Position

static:

This is the default value for all HTML elements. Elements with position: static are rendered in the normal document flow and are not affected by top, right, bottom, or left properties.

relative:

An element with position: relative is positioned according to the normal document flow, but its final position can be adjusted using top, right, bottom, and left. These offsets are relative to the element's original position in the flow. The space occupied by the element in the document flow remains, even if the element is visually shifted.

absolute:

An element with position: absolute is removed from the normal document flow and positioned relative to its closest positioned ancestor (an ancestor with a position value other than static). If no such ancestor exists, it's positioned relative to the initial containing block (usually the <html> element). Absolute positioning allows precise placement using top, right, bottom, and left.



# CSS Position

fixed:

Similar to absolute, an element with position: fixed is removed from the normal document flow. However, it is always positioned relative to the viewport and remains in the same place even when the page is scrolled. This is commonly used for elements like navigation bars that should always be visible.

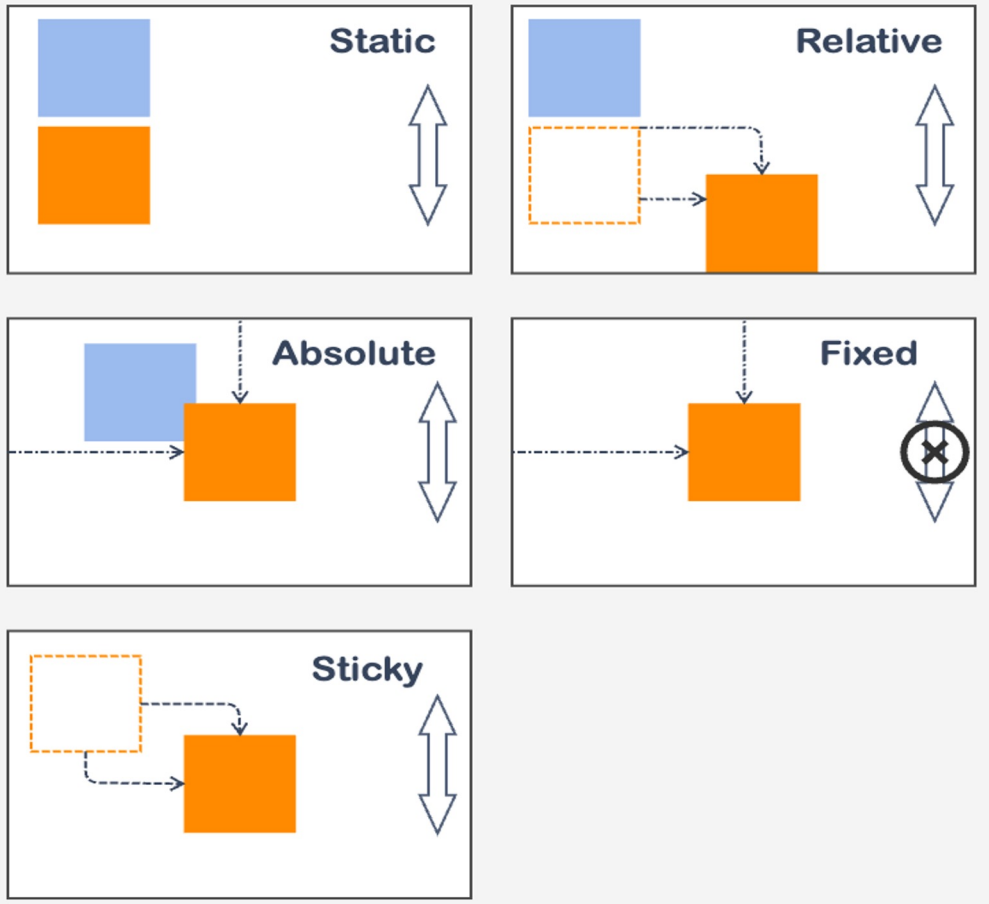
sticky:

This is a hybrid of relative and fixed positioning. An element with position: sticky behaves like a relative element until it reaches a specified scroll position within its containing block (defined by top, right, bottom, or left). Once that threshold is met, it becomes fixed relative to the viewport, effectively "sticking" in place during scrolling.

Anjah



# CSS Position




Go to Full Example
https://codepen.io/editor/Amjad-Ataallah/pen/01a0e8f8-4a68-7606-b54b-a9b8a752fd83

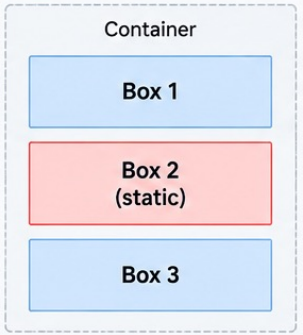
# CSS Position

## CSS Position

### static

Default value. Element is positioned in the normal document flow. Not affected by top, right, bottom, or left.

```
.box {
   position: static;
}
```

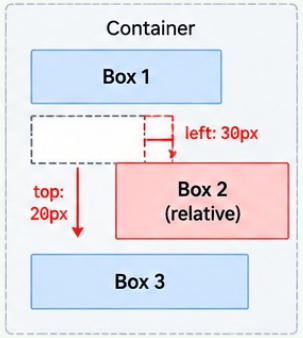



- Follows normal document flow.
- top, right, bottom, left have no effect.

### relative

Positioned according to the normal document flow, but can be moved using top, right, bottom, and left. The space remains in the document flow.

```
.box {
   position: relative;
   top: 20px;
   left: 30px;
}
```

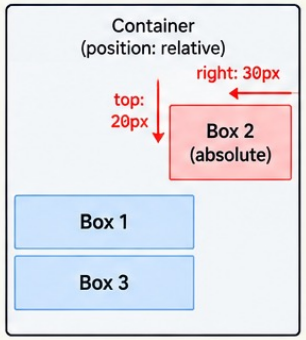



- Moved from its original position.
- Space in the document flow remains.

### absolute

Removed from normal document flow and positioned relative to its closest positioned ancestor (an ancestor with position other than static).

```
.box {
   position: absolute;
   top: 20px;
   right: 30px;
}
```




- Removed from normal document flow.
- Positioned relative to the nearest positioned ancestor.

### fixed

Removed from normal document flow and positioned relative to the viewport. Stays in the same place even when the page is scrolled.

```
.box {
   position: fixed;
   top: 20px;
   right: 30px;
}
```

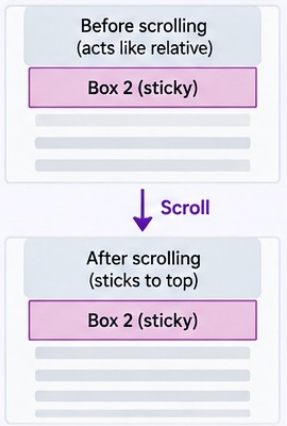



- Removed from normal document flow.
- Positioned relative to the viewport.
- Stays in the same place during scrolling.

### sticky

Acts like relative until it reaches a specified scroll position, then becomes fixed relative to the viewport.

```
.box {
   position: sticky;
   top: 0;
}
```




- Behaves like relative until the specified scroll position.
- Then sticks in place (like fixed).

# CSS z-index

The z-index CSS property controls the stacking order of positioned elements along the z-axis, which is the imaginary axis extending out from the screen. When elements overlap, z-index determines which element appears on top.

**Key points about z-index:**

**Requires position property:**

z-index only works on elements that have a position value other than static. This means you need to set position to relative, absolute, fixed, or sticky for z-index to have an effect.

**Numerical values:**

z-index accepts integer values (positive, negative, or zero).

**Stacking order:**

Elements with a higher z-index value will appear in front of elements with a lower z-index value within the same stacking context.

**Default behavior:**

If no z-index is specified, the default stacking order is determined by the document source order, meaning later elements in the HTML code will appear on top.

Go to Full Example
https://codepen.io/editor/Amjad-Ataallah/pen/01a0f70d-6122-7e3f-ab1e-9c566bc94e1a

Amjad

# HTML Containers & CSS

HTML containers are elements used to group and organize content, allowing CSS to control their appearance, layout, and pos itioning.

**Common Container Elements**

| Element | Description | Display Type |
|---|---|---|
| `<div>` | Generic container for grouping elements | Block |
| `<span>` | Generic container for styling parts of text | Inline |
| `<section>` | Groups related content into a thematic section | Block |
| `<article>` | Represents independent, self-contained content | Block |
| `<header>` | Contains introductory content or navigation | Block |
| `<footer>` | Contains footer information | Block |
| `<nav>` | Groups navigation links | Block |

Remember: HTML containers organize content, while CSS controls how those containers look and behave. CSS can also change their default display behavior using the display property.



# CSS Layout

CSS layout refers to the methods and techniques used to arrange and position elements on a web page. It dictates how content is displayed, its relationship to other elements, and its overall structure within the browser window.

CSS offers multiple layout techniques:

- Old: Float, Inline-block.
- Modern: Flexbox (1D), Grid (2D).

**Best Practice:** Use Flexbox for components and Grid for full-page layouts.

Amjad





# CSS Flexbox

Flexbox, or the Flexible Box Layout module, is a one-dimensional CSS layout model designed to arrange items within a container in either a row or a column. It simplifies the process of creating responsive and dynamic layouts by providing efficient ways to distribute space and align items, eliminating the need for older techniques like floats or complex positioning.

**Flexbox helps to accomplish the following:**

1. Horizontal or vertical alignment of elements
2. Arrangement (order) of elements
3. Space distribution between elements
4. Space distribution around elements

Amjad





# CSS Flexbox

**Notes to Remember**

1. Flexbox is one-dimensional. It can arrange items either in a row, or a column, but not both at the same time.
2. Flexbox is useful for laying out elements such as navigation bars, headers, image galleries, etc.
3. Flexbox fixes CSS layout madness by placing the responsibility for layout with the container, and not asking each item to be responsible for laying itself out

Amjad



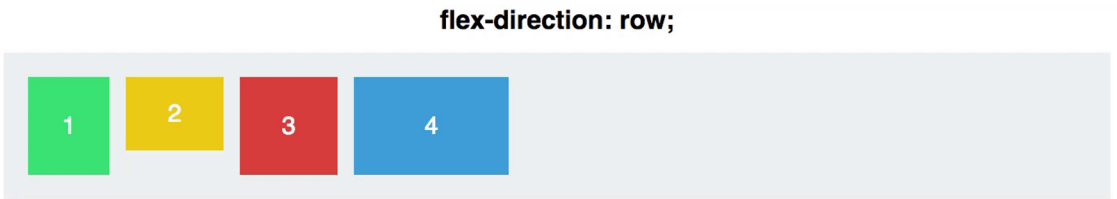
# CSS Flexbox Container

Flexbox works by applying CSS properties to both the container, and the children inside the container.

- First: add the CSS property `display:flex;` to the container of the items you intend to arrange.
- Remember, Flexbox is one dimensional. `display:flex` arranges your items in a row. This can be changed with `flex-direction: column`

```
<div class="container">
  <div>Item 1</div>
  <div>Item 2</div>
  <div>Item 3</div>
</div>
```

```
.container {
  display: flex;
  flex-direction: row;
}
```



# CSS Flex Direction

As we covered earlier, your flex items can be arranged either in a row, or a column. This is done with either `flex-direction:column` and `flex-direction:row`

```
#container {
  display: flex;
  flex-direction: row;
}
```



# CSS Flex Direction – Reversing Order

- Until now, re-arranging items with pure CSS was not possible.
- With Flexbox, you can reverse the order of your row or column with a single directive!
- On the flex container, use
  a. flex-direction: column-reverse;
  b. or flex-direction: row-reverse

```
.container {
  display: flex;
 flex-direction: column-reverse;
}
```

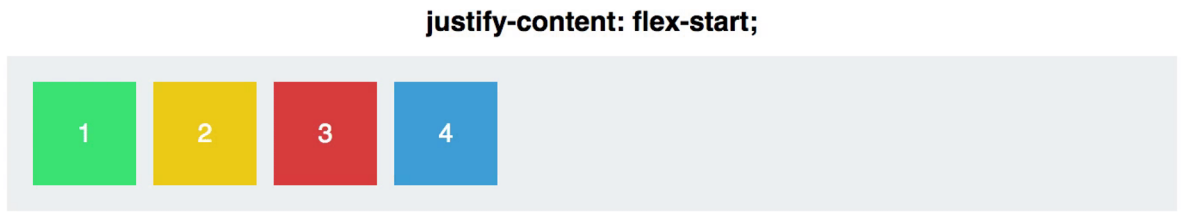


# CSS Flex Justify Content

Justifying your content, both horizontally and vertically, used to be a very large pain in the neck.

Not anymore! Flexbox makes this easy, with one simple line of CSS. There are 5 different values for the CSS property justify-content:

- Flex-start
- Flex-end
- Center
- Space-between
- Space-around



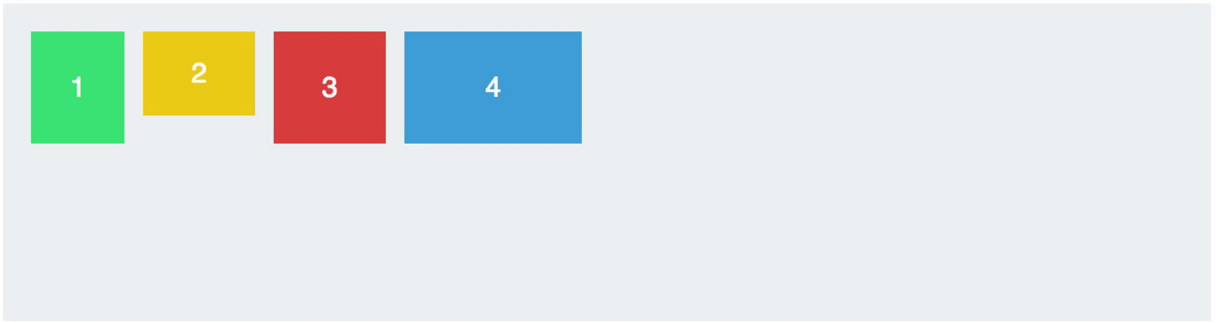

```
.container {
  display: flex;
  justify-content: center;
}
```

# CSS Flex Algin Items

align-items is also applied to the flex container. This is similar to justify content, but works along the cross-axis. If your items are arranged in a row, this would act on the vertical axis.

- Align items allow you to fill the container vertically, or align it along the baseline of the flex container (bottom edge). Before Flexbox, this was very difficult to do.
- The five values for align-items are:
  1. flex-start
  2. flex-end
  3. center
  4. stretch
  5. baseline

**align-items: flex-start;**




```
#container {
  display: flex;
  align-items: stretch;
}
```

# CSS Flex Order

This is a fairly simple property that accepts integer values for ordering items.

Want to switch the first two navigation items on your website? First the container must have the property display: flex;.

Then give the first item order: 2, and the second item order: 1.

***(Note that the default value is 0, so all items will need to be given an order number.)***

```
.firstItem {
  display: flex;
  order: 1;
}

.secondItem {
  display: flex;
  order: 2;
}
```

# CSS Flex Order

The **order** property is used to control the order of flex items. It accepts integer values. Items with a lower order value appear first. The default value is 0.

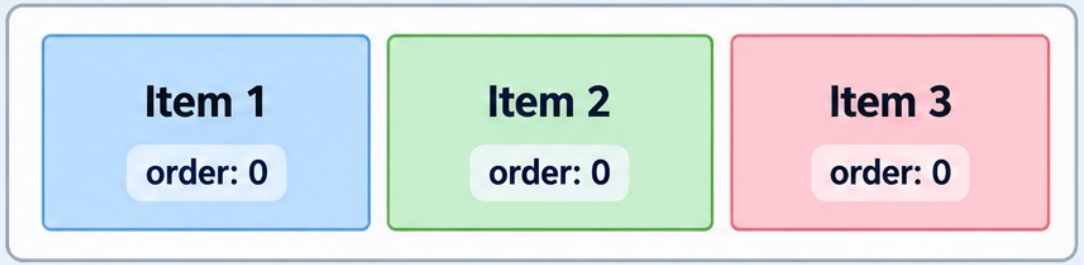
**HTML**

```html
<div class="container">
   <div class="firstItem">Item 1</div>
   <div class="secondItem">Item 2</div>
   <div class="thirdItem">Item 3</div>
</div>
```

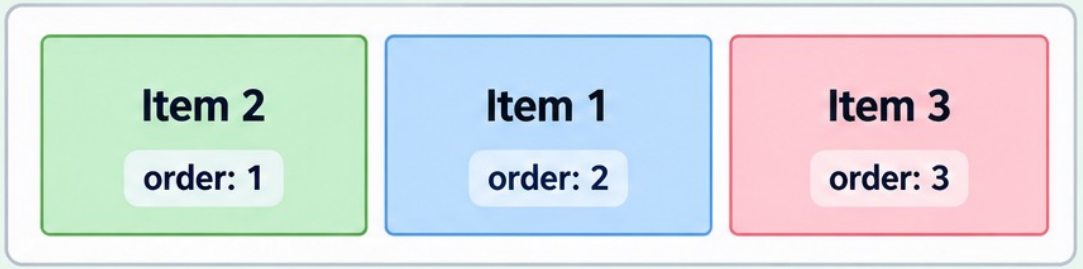
**CSS**

```css
.container {
   display: flex;
}
.firstItem {
   order: 2;
}
.secondItem {
   order: 1;
}
.thirdItem {
   order: 3;
}
```

**Default order (all items order: 0)**




**After changing the order**



# CSS Flex Align Self

The **align-self** property is used to align an individual flex item. It accepts the same values as **align-items**, and achieves the same results, but for a single item.

**HTML**

```html
<div class="container">
  <div class="item item1">1</div>
  <div class="item item2">2</div>
  <div class="item item3">3</div>
  <div class="item item4">4</div>
  <div class="item item5">5</div>
</div>
```

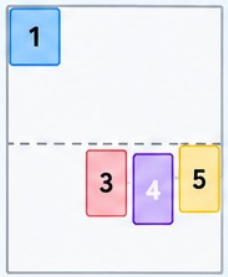
**CSS**

```css
.container {
  display: flex;
  height: 120px;
  border: 2px solid #ccc;
  align-items: center; /* default */
}
.item1 {
  align-self: flex-start;
}
.item2 {
  align-self: flex-end;
}
.item3 {
  align-self: center;
}
.item4 {
  align-self: stretch;
}
.item5 {
  align-self: baseline;
}
```

## Align right items with align-self (container has align-items: center)

### flex-start

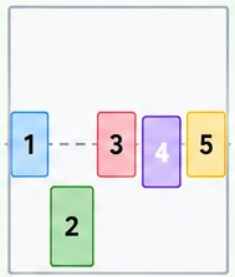
```css
.item1 {
  align-self: flex-start;
}
```




Aligns the item to the start of the cross axis (top).

### flex-end

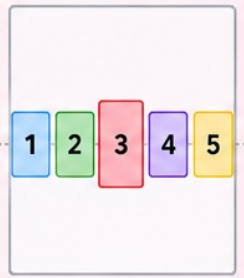
```css
.item2 {
  align-self: flex-end;
}
```




Aligns the item to the end of the cross axis (bottom).

### center

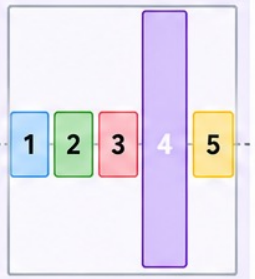
```css
.item3 {
  align-self: center;
}
```




Aligns the item to the center of the cross axis.

### stretch

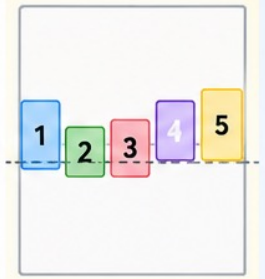
```css
.item4 {
  align-self: stretch;
}
```




Stretches the item to fill the container along the cross axis.

### baseline

```css
.item5 {
  align-self: baseline;
}
```




Aligns the item based on the text baseline.

# CSS Flex Grid (Flex inside Flex)

You can create a simple grid layout by putting a Flexbox inside a Flexbox.
The outer container uses flexbox to create columns, and each column uses flexbox to arrange its items.

**HTML**

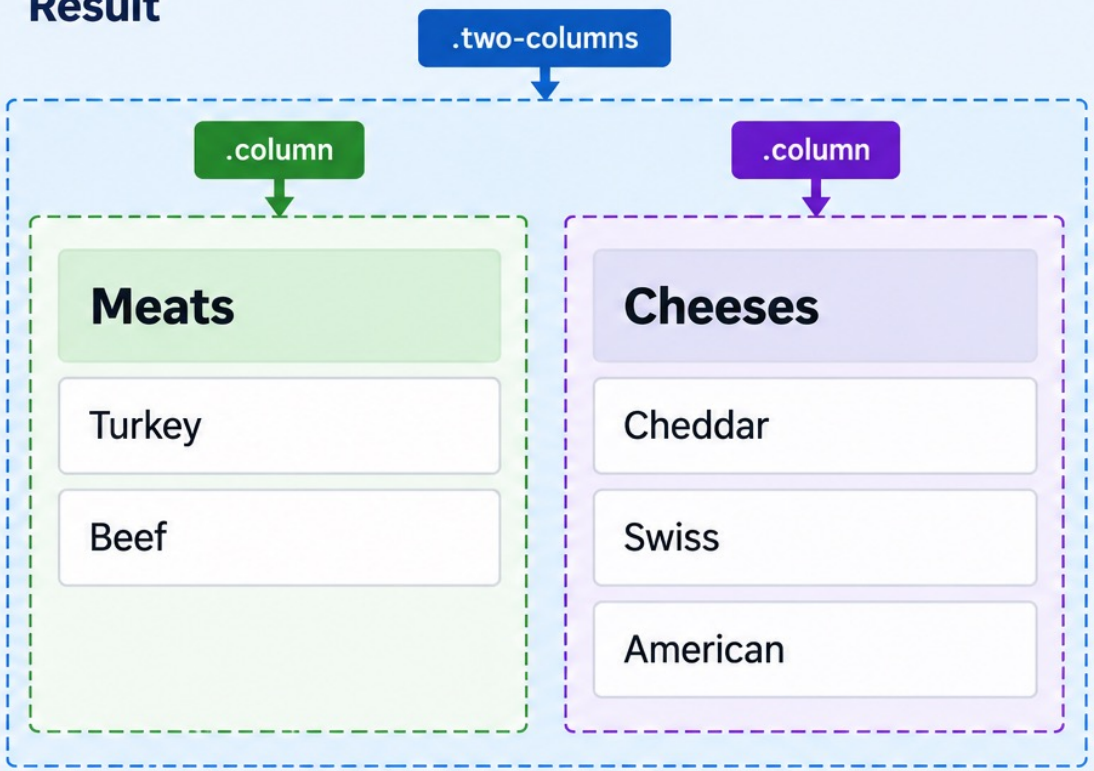
```html
<div class="two-columns">
  <div class="column">
    <h2>Meats</h2>
    <div>Turkey</div>
    <div>Beef</div>
  </div>
  <div class="column">
    <h2>Cheeses</h2>
    <div>Cheddar</div>
    <div>Swiss</div>
    <div>American</div>
  </div>
</div>
```

**CSS**

```css
.two-columns {
  display: flex;
  flex-direction: row;
}

.column {
  display: flex;
  flex-direction: column;
}
```

**Result**





# CSS Flex (Useful Links)

- Flexbox Froggy - a step-by-step test-driven game that teaches you flexbox
- Flexbox Defense
- How Flexbox Works With Animated Images
- Learn CSS Flexbox in 5 Minutes
- https://css-tricks.com/snippets/css/a-guide-to-flexbox
- https://yoksel.github.io/flex-cheatsheet/#align-items
- https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Flexbox
- https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Flexible_Box_Layout

Anjad



# CSS Responsive Design

- **Responsive Web Design** allows a website to **adapt its layout and styling to different screen sizes**, such as desktops, tablets, and mobile phones.
- CSS uses **Media Queries** to apply styles only when specific conditions are met.



0-480

Smaller smartphones



481-768

Tablets & larger smartphones



769-1279

Laptops, larger tablets in landscape, and small desktops



1280+

Larger desktops and monitors

# CSS Responsive Design

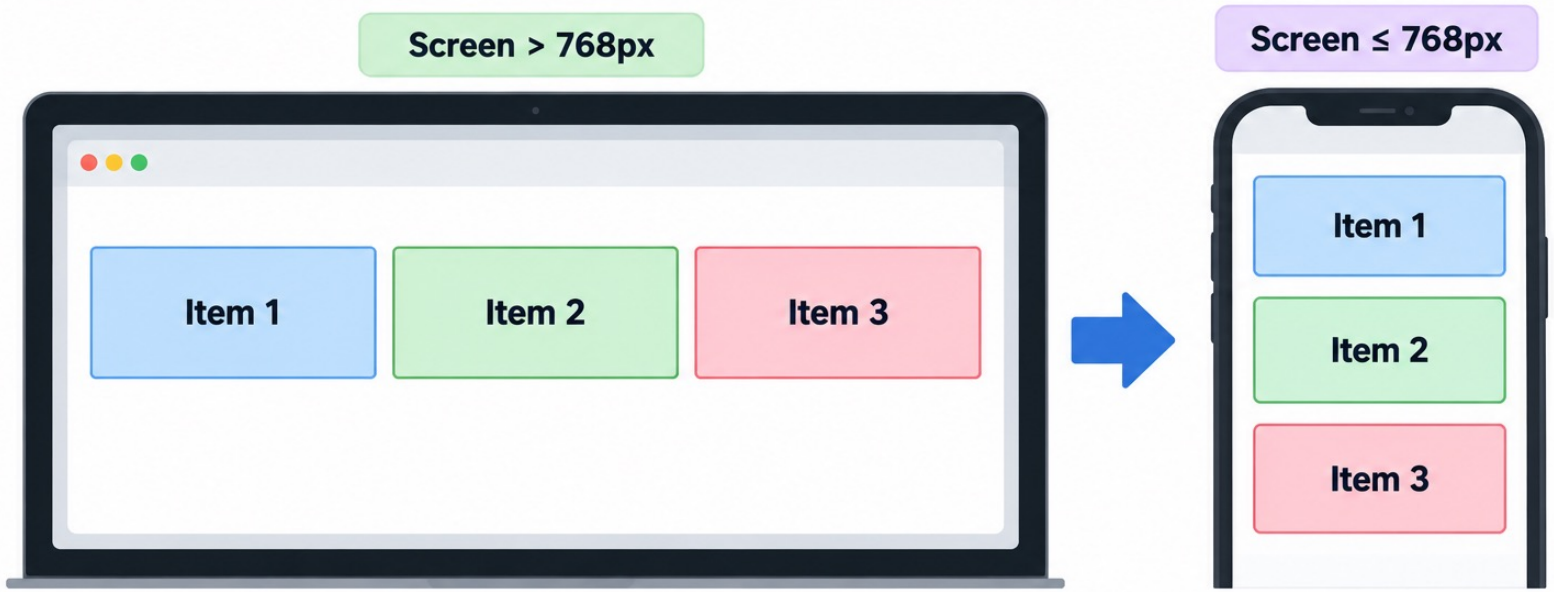
## Media Query Syntax

```
@media screen and (max-width: 768px) {
    /* CSS for smaller screens */
}
```

## Example: Mobile Layout

```
.container {
  display: flex;
  flex-direction: row;
}

@media screen and (max-width: 768px) {
    .container {
      flex-direction: column;
    }
}
```





# CSS Frameworks & Libraries

A **CSS framework/library** provides **pre-written CSS styles, components, and layout tools** that help developers build websites faster without styling everything from scratch.

**Popular CSS Frameworks**:

| Framework | Description |
|---|---|
| **Bootstrap** | Ready-made components and responsive grid system |
| **Tailwind CSS** | Utility-first framework using small reusable CSS classes |
| **Bulma** | Lightweight CSS framework based on Flexbox |
| **Foundation** | Responsive front-end framework for websites and applications |

Instead of writing your own CSS, frameworks provides predefined classes:

Ariiq

# CSS Best Practices

CSS allows developers to combine multiple classes, reuse styles, and write clean, maintainable code.

**1. Multiple Classes on One Element:** An HTML element can have multiple CSS classes, separated by spaces.

```html
<button class="btn primary large">Save</button>
```

```css
.btn {
    padding: 10px;
    border-radius: 5px;
}

.primary {
    background-color: blue;
    color: white;
}

.large {
    font-size: 20px;
}
```

**All three classes are applied to the same button.**

Amjad

# CSS Best Practices

**2. Combining Classes in CSS Selectors:** Selects elements that have both btn and primary classes.

```
.btn.primary {
    background-color: blue;
}
```

Selects elements that have **both** `btn` and **primary** classes.

```
.btn .primary {
    background-color: blue;
}
```

Selects elements with class **primary** **inside** an element with class **btn**.

**Important**: The space changes the meaning of the selector.

Amjad



# CSS Best Practices

### 3. Reuse Classes Instead of Repeating CSS

One CSS class can style multiple elements.

```
<p class="highlight">First paragraph</p>
<p class="highlight">Second paragraph</p>
```

```
.highlight {
    color: red;
    font-weight: bold;
}
```

Amjad



# CSS Best Practices

**4. Class vs. ID**

| **Class (.)** | **ID (#)** |
|---|---|
| Can be reused across elements | Should be unique within a page |
| An element can have multiple classes | An element should have only one ID |
| Preferred for reusable styling | Useful for unique identification and page anchors |

Amjad

# CSS Best Practices

## 5. Use Meaningful Class Names

**Avoid using:**

```
.redText {
    color: red;
}
```

**Prefer using:**

```
.error-message {
    color: red;
}
```

**Names should describe the purpose of an element rather than only its appearance**

Amjad

# CSS Best Practices

## 6. Avoid Inline Styles for Reusable Design

**Avoid using:**

```
<p style="color: red;">Error!</p>
```

Prefer using:

```
<p class="error-message">Error!</p>
```

```
.error-message {
    color: red;
}
```

Amjad

# CSS Best Practices

### 7. Understand the CSS Cascade

When multiple CSS rules target the same element, the browser determines which declaration wins based on factors including **importance, specificity, and source order**.

```html
<p class="message">Hello World</p>
```

```css
p {
    color: black;
}

.message {
    color: blue;
}
```

**Result**: The text is blue because the class selector has higher specificity than the element selector.

**Key takeaway:** Use reusable classes, meaningful names, external stylesheets, and simple selectors to keep CSS clean and maintainable.

Amjad

# Author


## Amjad Ataallah

Email: amjad.atallah@najah.edu
Linkedin: https://www.linkedin.com/in/ataallah/
Website: https://amjadataallah.github.io

## Yanal Sweseh

Email: yanal.swaseh@najah.edu
Linkedin: https://www.linkedin.com/in/yanalsweseh/
Website: https://github.com/yswesieh